

Wind turbine power curve modeling using radial basis function neural networks and tabu search

Despina Karamichailidou, Vasiliki Kaloutsou, Alex Alexandridis *

Department of Electrical and Electronic Engineering, University of West Attica, Ancient Olive Grove Campus, Thivon 250 & P. Ralli, 12244, Aigaleo, Greece

ARTICLE INFO

Article history:

Received 18 July 2019

Received in revised form

8 August 2020

Accepted 5 October 2020

Available online 8 October 2020

Keywords:

Artificial neural networks

Fuzzy means algorithm

Radial basis function

Tabu search

Wind energy

Wind turbine power curve

ABSTRACT

Wind turbine power curve (WTPC) modeling is of great importance for performance monitoring. This work proposes a new method for producing highly accurate non-parametric models for wind turbines based on artificial neural networks (ANNs). To achieve this, we employ networks belonging to the radial basis function (RBF) architecture, and feed them with additional important input variables besides wind speed. To further increase modeling accuracy, while at the same time keeping the computational cost at acceptable levels, we introduce a new training algorithm based on the successful non-symmetric fuzzy means (NSFM) approach, which in this work is hybridized with the tabu search (TS) metaheuristic technique, enabling the method to train efficiently datasets of high dimensionality. The resulting method is evaluated on real data from four wind turbines, whereas a comparison with numerous WTPC modeling schemes, including parametric and non-parametric models is conducted. The solution found by the proposed algorithm outperforms the results produced by its rivals in terms of both modeling accuracy and efficiency, while in most cases it also leads to simpler models. The resulting models can be used successfully, not only for accurate WTPC modeling, but also for constructing wind turbine performance analysis tools, e.g. 3-D power curves.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

Energy is essential for economic and social development; on the other hand, it poses an environmental challenge as the global population grows and therefore all aspects of energy production and consumption should be addressed [1]. The urgent need to reduce the pollutant emissions that come as byproducts of energy production and the finiteness of the earth's fossil fuel reserves, make renewable energy to be part of the solution for the energy problem [2]. Since antiquity, mankind has been using wind power, but its reappearance as an important source of energy began at the end of the 20th century as a notable development; indeed, nowadays, wind energy appears to be one of the cost-effective options for electricity generation [1,2].

The theoretical power output (P) from a modern wind turbine (WT) is given by the following expression:

$$P = \frac{1}{2} C_p(\lambda, \beta) \rho A v^3 \quad (1)$$

where $C_p(\lambda, \beta)$ is the power coefficient, β and λ are the blade pitch angle and tip-speed ratio, respectively, ρ is the density of air, A is the rotor swept area and v is the wind speed [3]. The produced power depends mainly on the wind speed, exhibiting a cubic relationship, but is also affected by other factors, like the characteristics of the wind turbine, e.g. rotor dimensions, and the rest of the weather conditions, e.g. air temperature and wind direction.

The electrical power output of a wind turbine as a function of the wind speed is shown in a graph, known as the power curve [2]. Given a typical wind turbine, the power curve (WTPC) consists of three key wind speed points: (i) the cut-in speed, i.e. the minimum speed of wind so that the turbine is able to generate power, (ii) the rated wind speed, i.e. the wind speed necessary for reaching the rated power and (iii) the cut-out speed, i.e. the maximum wind speed for which the wind turbine is allowed to generate power. At high values of wind speed, the produced power is limited by engineering design, while there are three strategies that are used to control the speed of the wind to prevent from damaging the wind turbine: pitch, yaw and stall control [2]. Modern wind turbines are pitch-controlled, i.e. power output is kept constant at wind speeds above the rated value [4].

The theoretical power curve of a wind turbine is given by the

* Corresponding author.

E-mail address: alex@uniwa.gr (A. Alexandridis).

Abbreviations

WT	wind turbine
WTPC	wind turbine power curve
CI	computational intelligence
ANN	artificial neural network
RBF	radial basis function
RBFN	radial basis function network
FM	fuzzy means
SFM	symmetric fuzzy means
NSFM	non-symmetric fuzzy means
TS	tabu search
TL	tabu list
AC	aspiration criterion
MAD	median absolute deviation
MAE	mean absolute error
R^2	coefficient of determination
P	wind turbine power output (kW)

C_p	power coefficient
λ	tip-speed ratio
β	blade pitch angle
ρ	density of air (kg/m^3)
A	rotor swept area (m^2)
v	wind speed (m/s)
<i>RBF network parameters</i>	
w_j	synaptic weight of the j -th RBF unit
$\mathbf{x}$	input vector
y	data output
$\hat{y}$	output of the network
ϕ_j	RBF of the j -th hidden unit
μ_l	activity of the l -unit
$h_{j,i}$	output of the j -th hidden unit to the i -th input
s_i	number of fuzzy sets in i -th dimension

manufacturer and it is acquired assuming an ideal situation, where the actual field conditions, such as local orography and weak effects produced by other wind turbines are ignored [3]. Therefore, the actual power curve deviates from the standard power curve, because the latter does not take into account important variables associated with the produced power output of a wind turbine, e.g. atmospheric pressure, turbulence intensity and wind direction variability, among others [5].

Wind turbine power curve modeling is an essential tool in turbine performance monitoring and power prediction or forecasting [6], and therefore developing accurate models for WTPC is considered as a very important area of research [7]. Moreover, the power curve of a wind turbine helps in obtaining a rough wind energy estimation without taking into account technical details of its generating system [2]. There are several statistical techniques to fit the power curve of a wind turbine to data from real-world operation, which can be classified into parametric and non-parametric methods [6–8]. To define the relationship between input variables and output response in a parametric model, a set of mathematical equations consisting of a finite number of parameters is assumed. Though parametric equations may contain free parameters that can be fitted to actual data, they have a fixed form, which is based on the underlying physical laws. A non-parametric approach, on the other hand, makes no a priori assumptions regarding the form of the governing equations, which in this case is determined by the available data [9]. Notwithstanding their name, non-parametric models also make use of parameters, but, in contrast to parametric models, the number and nature of these parameters are flexible and not fixed in advance [7].

Although the shape of the wind turbine power curve has, in some cases, encouraged the choice of parametric models, their application is associated with certain limitations [6], as they are usually complex to use, while they are considered to be less accurate in describing the actual behavior of real-world wind turbines [10]. Non-parametric models, on the other hand, appear to be more flexible, and thus more precise, in terms of accurately modeling power curves [6], their main drawback being the increased computational cost compared to parametric ones. In what follows, a review of recent literature is presented, while a comprehensive list of WTPC modeling techniques can be found in Ref. [9].

In Ref. [6], WTPC is modelled using advanced parametric and non-parametric methods. The results of this study suggest that penalized spline regression presents a better performance over

other analyzed methods. In Ref. [11], parametric models are developed for WTPC, based on non-linear regression functions. The models take into account the wind speed, while the wind direction impact on power curve is tested. Wind farm power curve models have also been published in literature; e.g. in Ref. [12], wind farm performance monitoring is accomplished by using three different operational curves.

More elaborate non-parametric methods have been applied to wind turbine systems (e.g. predicting the generated wind power in wind farms), involving computational intelligence (CI) techniques [13], which can be more efficient and accurate than traditional statistical methods [14]. CI consists of nature-inspired computational tools, such as evolutionary computation, swarm intelligence, fuzzy logic and artificial neural networks (ANNs), which have been used extensively to solve a wide range of complex problems. ANNs constitute an important tool in the field of CI that tries to simulate some properties of biological neural networks as found in the human brain, through a “black-box” approach; ANNs can model an unknown system without any prior knowledge for its behavior, based solely on input and output data [15]. In the next paragraph, works of literature dealing with CI models for WTPC are discussed briefly, whereas a survey of ANNs in wind energy systems is given in Ref. [16].

In Ref. [7], a differential evolution algorithm has been employed for the estimation of a five-parameter logistic function for WTPC modeling. Non-parametric models have been developed as well, including ANN, fuzzy c-means clustering and data mining techniques using the wind speed as input variable. The results are favorable for the parametric model, while the ANN is claimed to be the best nonparametric model. In Ref. [17], parametric and non-parametric methods, including multi-layer perceptron (MLP) networks and genetic algorithms, are used to build models for power prediction that can be used for on-line monitoring of wind farms. The k-nearest neighbor model results in a good performance for the wind farm operating in normal conditions. In Ref. [18], different non-parametric methods based on various neural network architectures are constructed for WTPC modeling; the produced models can also be used to estimate the reference power curve for on-line monitoring of wind farm performance. In Ref. [19], non-parametric models, including ANNs, are developed for power curve monitoring; it is shown that when the ambient temperature and wind direction are considered as input parameters along with wind speed, the prediction errors are reduced, while the model is capable

of detecting faults earlier. In Ref. [20], an artificial neural network multistage model is constructed to model WTPC incorporating six input parameters of the wind characteristics. It is claimed that this method gives better results compared to different parametric and non-parametric methods. In Ref. [14], an ANN model with three inputs for wind power forecasting is proposed, leading to accurate predictions. In Ref. [21], a support vector machine algorithm with three kernel functions is used to build power curve models, which are compared with models published in literature; results show that there exists a data partition which can lead to satisfactory performance. In Ref. [22], an ANN with two inputs is constructed, and, combined with gaussian process filtering, it leads to an accurate method for calculating wind direction-dependent power curves. In Ref. [23], a comparison of modeling methods for wind power prediction is conducted, where various parametric and non-parametric models, including ANNs are developed for power curve modeling. The study of the effect of input parameters in wind power prediction is the main contribution of this work. The prediction results are favorable for the ANNs, as they come with the lowest values of metric errors, due to their inherent universal approximation characteristic. In Ref. [24], an MLP network with several inputs is trained and optimized using a genetic algorithm to model WTPC; the proposed ANN model achieves a satisfactory performance when compared to Fourier series interpolation, as it takes into account simultaneously up to ten important climatic variables that influence power output.

The contribution of the above recently-published works is essential, as the estimations of neural network models produced by well-known and tested training algorithms are improved in conjunction with either specific filtering procedures, or parameter optimization techniques. However, the latter usually increase considerably the computational cost, which is already high due to the data-driven nature of non-parametric approaches, thus impeding their use in large-volume datasets; moreover, further improvement in terms of modeling accuracy and/or computational efficiency could be achieved by employing novel, non-standard training algorithms, or even different ANN architectures compared to standard MLPs.

Radial basis function (RBF) networks are a special type of feed-forward neural networks that consist of a single hidden layer [25]. Their simple structure provides several important advantages regarding speed and efficiency when compared to different popular architectures, like MLPs, while they are also characterized by the universal approximation property, robustness, good generalization and very competitive results compared to other machine learning approaches [26]. RBF neural networks have been successfully applied to wind energy systems [16]; for example, in Ref. [27], three different ANN architectures, including RBFNs, are employed for short term wind speed forecasting. In Ref. [28], an improved RBF neural network with an error feedback scheme is developed for short-term wind speed forecasting, proposing a parameter initialization and introducing a shape parameter in the Gaussian basis function.

Surprisingly, despite the significant advantages offered by the RBF architecture, only a small number of published works employ them for wind power estimation. In Ref. [29], an RBF neural network is trained for wind power prediction and appears to be superior to MLP networks. Earlier attempts concern classic training algorithms, as for example in Ref. [4], where fuzzy modeling techniques and ANNs are used to estimate the annual energy output of a wind turbine. The RBF network in this study comes with a pre-fixed hidden layer size and makes use of Gaussian kernels. In a recent work [30], an RBF neural network is developed for wind power prediction, using a hybrid particle swarm optimization (PSO)-based fuzzy c-means clustering procedure and extreme learning machines.

In this paper, the benefits of the RBF architecture are exploited in order to build a new non-parametric method for modeling the power curve of wind turbines with increased accuracy. The method allows to take into account large amounts of data and multiple inputs that affect power generation besides wind speed. The network parameters are determined automatically through a novel training procedure based on the non-symmetric fuzzy partitioning (NSFM) algorithm of the input space [31]. Variants of this algorithm have been shown to offer increased accuracy in different applications, but require tedious search procedures for parameter optimization, which involve either exhaustive search [32], or population-based metaheuristic search methods [31,33]; such approaches impose a high computational burden and coupling them with a non-parametric training procedure - which is by itself computationally demanding - may render the scheme impractical for use on large datasets. An important contribution of this work, concerns the introduction of a modified tabu search (TS) metaheuristic technique [34], which is wrapped around the NSFM algorithm; the proposed approach is not population-based, but rather acts on a single solution, thus helping to obtain the network parameters at an acceptable computational cost, even when using high-volume datasets, and ultimately rendering the method suitable for producing highly accurate models for WTPC modeling.

The outline of this paper is as follows: in section 2 a description of the available data is given; section 3 presents an introduction to artificial neural networks, which is followed by a description of the RBF network architecture and the fuzzy means training algorithm; in section 4 standard tabu search is presented followed by a detailed description of the proposed method; in section 5 the case study is described and the results are presented, whereas section 6 summarizes the conclusions of the current study, and provides plans for future work.

2. Data description

The proposed approach for power curve modeling was evaluated on data from four different pitch-controlled wind turbines belonging to the same wind farm. The location of the wind farm is not disclosed due to confidentiality reasons. The collected data correspond to a time period of 1 year and were recorded in 10-min intervals, amounting to a total of approximately 50k datapoints for each wind turbine. Wind data are measured by nacelle anemometers.

Scatter power curves based on the raw data for the four wind turbines are depicted in Fig. 1. As it can be seen, the plots deviate from the typical form of the power curve obtained in standard conditions by manufacturers. This is expected, as typically power curves contain abnormal values that are characterized as outliers: wind speed values close to the cut-in or the cut-out speed, negative power values, non-operational periods, invalid, missing or corrupted data due to sensor and pitch malfunction, dirt or icing on blades, etc. [35]. To this end, a filtering process is used to filter out erroneous data in order to eliminate the impact of different operating conditions. More details about the filtering procedure followed in this work can be found in section 5.1.

As soon as datasets are filtered out and cleared from outliers, a model for power curve approximation can be developed based on qualitative data. In order to increase estimation accuracy, the data-driven models produced in this work make use of four additional input variables besides wind speed, which have been shown in the literature to have a strong influence on the generated power, namely the wind direction and ambient temperature [19], and the blade pitch angle [23,36].

More precisely, wind direction fluctuations, and hence turbulence intensity variations due to field conditions such as

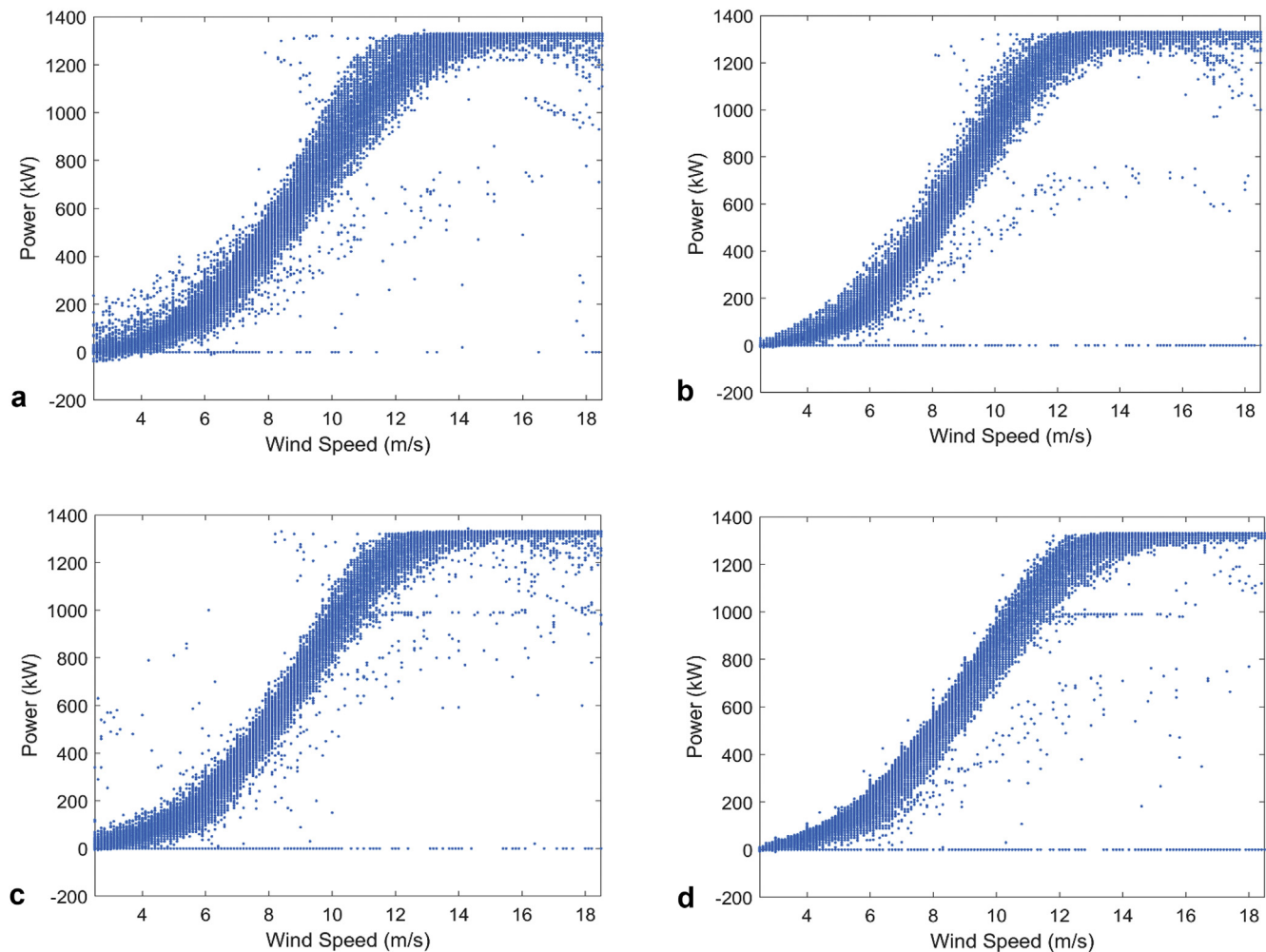


Fig. 1. Actual wind turbine power output as a function of wind speed for: (a) wind turbine no. 1, (b) wind turbine no. 2, (c) wind turbine no. 3 and (d) wind turbine no. 4.

topography and wake effects of neighboring wind turbines, have a significant effect on power output. The air density is mainly influenced by the ambient temperature, and thus the latter also has a significant impact on the generated power. Blade pitch angle affects the aerodynamic efficiency and hence the output power, by changing the angle of attack of the wind. The particular set of variables are selected as inputs in this case, so as to further improve the estimation performance of the models. However, depending on the type of wind turbines and other case study specifics, there may be different factors that directly affect the power output; it is important to stress the fact that, in contrast to parametric models, the proposed approach can trivially accommodate for any additional input variable deemed necessary to improve modeling accuracy.

The relationship between the particular set of input variables and the generated turbine power, is expected to be non-linear, whereas its increased complexity makes it difficult to formulate an exact equation taking into account all the aforementioned factors; this leads us to the use of non-parametric models, focusing on artificial neural networks, which have been shown to be a promising approach for WTPC modeling [20].

3. Artificial neural networks

3.1. Introduction

Artificial neural networks constitute one of the most important computational intelligence tools and are used in a wide variety of problems, including function approximation and pattern recognition [25]. As their name implies, ANNs are inspired by biological neural networks; leaving aside complicated biological terms, each neuron in a biological neural network receives an input and transmits an output signal to all the connected neurons, according to the amount of its activation. Correspondingly, an ANN consists of many computational units which try to imitate this operation, called artificial neurons, or more simply, neurons. Each neuron receives an input and produces an output which depends on the value of a mathematical function, usually known as the activation function.

ANNs are characterized by many special properties [13]; perhaps the most important of them is learning through examples, a process called training. In short, during the training stage the ANN parameters are estimated, so that a specific input yields the

corresponding output found in the training examples. Another important ANN property is the ability of generalization, i.e. the ability of a network to perform well on previously unseen examples.

The way in which neurons in an ANN are interconnected defines the network architecture. The number and the type of neuron connectivity, as well as the activation function, are important parameters and their selection is determinant in order to produce an adequate network for a specific case study.

3.2. Radial basis function networks

Radial basis function networks (RBFNs) constitute an important ANN architecture, proposed independently by Broomhead, Lowe and other researchers at the end of the 80's [37]. The typical structure of RBF networks used in this work is illustrated in Fig. 2. RBFNs are feed-forward networks with one input layer, one hidden layer and one output layer. The input layer distributes the normalized input variables to the L hidden units of the hidden layer. Each hidden unit implements a radial basis function (hence the name of the network), associated to a center vector, with dimensionality equal to the number of input variables. The latter leads to a non-linear connection between the input and the hidden layer, while the output layer is linearly attached to the hidden layer [31].

Due to their simple structure, RBF networks present important advantages regarding speed and efficiency when compared to other popular architectures like MLP networks [31,38].

To formulate the training problem of an RBF network, a set of input-output data is assumed

$$(\mathbf{x}_i, y_i), i = 1, 2, \dots, K \quad (2)$$

where $\mathbf{x}_i \in \mathbb{R}^N$, $y_i \in \mathbb{R}$, N is the number of input variables, K is the total number of examples and L is the number of units in the hidden layer. The aim of the training procedure is to produce an approximating function $f: \mathbb{R}^N \rightarrow \mathbb{R}$, so that each input yields a corresponding target output. Considering Fig. 2, the hidden units implement an RBF function, called ϕ_j ($j = 1, 2, \dots, L$). This work makes use of the thin plate spline function:

$$\phi(v) = v^2 \log(v) \quad (3)$$

where $v_l(\mathbf{x}(k))$ is called the activity of the l -th unit and is defined as

$$v_l(\mathbf{x}(k)) = \|\mathbf{x}(k) - \mathbf{c}_l\|_2 = \sqrt{\sum_{i=1}^N (x_i(k) - c_{i,l})^2}, \quad k = 1, 2, \dots, K \quad (4)$$

where $\mathbf{c}_l$ is the center vector of the l -th RBF unit and $\mathbf{x}(k)$ is the k -th input vector.

The output of the j -th hidden unit to the i -th input is

$$h_{j,i}(\mathbf{x}_i) = \phi_j(\|\mathbf{x}_i - \mathbf{c}_j\|_2) \quad (5)$$

for every input example $i = 1, 2, \dots, K$.

The RBFN output for an example i is

$$\hat{y}_i(\mathbf{x}_i) = \sum_{j=1}^L w_j \phi_j(\mathbf{x}_i, \mathbf{c}_j) \quad (6)$$

where w_j is the synaptic weight of the j -th RBF unit.

The training procedure of an RBF network consists of calculating values for the following parameters: (i) number of RBF units, (ii) centers of RBF units, and (iii) synaptic weights. The number of RBF units and their corresponding center coordinates are crucial to the network performance [39], but once the RBF centers are fixed, the synaptic weights can be determined by solving a linear least squares problem, due to the linear connection between the hidden and the output layer, as follows:

$$\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y} \quad (7)$$

where Φ is a matrix containing the activities of every hidden unit to all input vectors, and $\mathbf{y}$ is a vector containing all the target values.

A popular approach for determining the locations of RBF centers is the k -means algorithm [40]. k -means clustering is by nature an iterative algorithm and does not have the ability for automatic

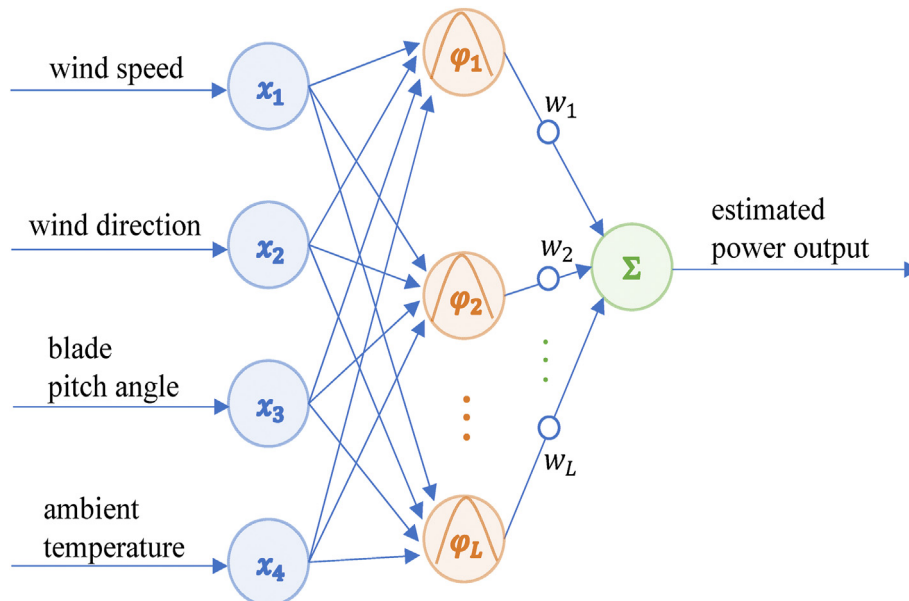


Fig. 2. An RBFN structure with 4 input variables, L hidden units and one output variable.

determination of the number of centers. To overcome the disadvantages of the aforementioned technique, this work adopts a variant of the fuzzy means (FM) algorithm [32], described briefly in the next section. It should be noted that the fuzzy means algorithm comes along with many successful applications [33,41–43] in diverse scientific fields.

3.3. Symmetric and non-symmetric fuzzy means algorithm

Contrary to the k -means algorithm, where clustering is conducted discretely and every datapoint belongs explicitly to one specific cluster, following the fuzzy logic approach [44] the datapoints may belong to more than one fuzzy set, according to a membership degree. To materialize the fuzzy means algorithm, therefore, which performs fuzzy partitioning of an N -D dimensional input space, a total of K datapoints are considered. Initially, the domain of each variable is partitioned into a number s_i ($i = 1, 2, \dots, N$) of triangular one-dimensional fuzzy sets. The combinations of fuzzy sets from each dimension produce the so-called fuzzy subspaces; the input space is partitioned into a total of S fuzzy subspaces, which is equal to the product of the number of fuzzy sets in each dimension. The algorithm then seeks a subset consisting of L fuzzy subspaces which are chosen so that they uniformly cover the distribution of the input space and whose centers will constitute the RBF network centers. This is accomplished through a non-iterative procedure, which leads to a fast RBF center selection process, even in the case of big datasets [26].

If the number of fuzzy sets is the same for every dimension ($s_1 = s_2 = \dots = s_N$), the partitioning is considered to be symmetric, which leads to the standard symmetric fuzzy means (SFM) algorithm. In this case, the only parameter to be chosen by the user is the number of fuzzy sets, which is common for all dimensions. This number affects the size of the network and thus, its estimation and generalization capabilities.

On the other hand, a non-symmetric partitioning (where $s_i \neq s_j$, $i \neq j$), could lead to RBF networks presenting greater flexibility, with the potential to achieve better estimation accuracy [31]. Implementing this non-symmetric partitioning leads to the non-symmetric fuzzy means (NSFM) algorithm, which is based on the same basic principles with standard SFM, with the difference that this time the user has to select N different parameters, i.e. the vector comprising the numbers of fuzzy sets, which are now different for each dimension.

For systems consisting of a low number of input variables N , this selection could be performed using a trial and error procedure where all the combinations of fuzzy sets per dimension are tested. However, in systems with large, or even moderate dimensionalities, an exhaustive search for all possible combinations may lead to very high computational times, making the use of NSFM algorithm impractical. Therefore, more efficient ways have to be incorporated, in order to find an adequate fuzzy partitioning of the input space with respect to network performance in a reasonable time.

In such cases where exact methods are too expensive to apply, one has to utilize more efficient ways in order to obtain sufficiently good, but possible not globally optimal, solutions. Within this context, the NSFM algorithm was successfully coupled with metaheuristic search methods belonging to evolutionary computation [33] and swarm intelligence [31]. Such techniques are based on a population of potential solutions, which are evolved over a number of iterations. Although these approaches can yield significant benefits when compared to the impractical exhaustive search procedure, they are still considered to be computationally expensive due to the need to maintain a population of potential solutions, which leads to a significant number of function evaluations per each iteration. The problem becomes more apparent for systems

with a large number of datapoints and/or input variables.

As the datasets involved in this work, which have been described in section 2, fall into this category, we introduce a new, less computationally demanding method for implementing the NSFM algorithm in order to produce accurate RBF networks for WTPC modeling. This is achieved by coupling the NSFM algorithm with a metaheuristic search method known as tabu search (TS) [34], which, in contrast to the aforementioned population-based methods, evolves only a single solution, and thus is more appropriate for large datasets.

4. Tabu search – non symmetric fuzzy means (TS-NSFM) algorithm

4.1. Standard tabu search

Along with different approaches that find application in the context of metaheuristic methods, such as genetic algorithms and particle swarm optimization, TS has also been designed to avoid being trapped in local optima by using heuristics to explore the solution space beyond local optimality. Its popularity has significantly grown during the last decades, as it is characterized as the most efficient method for handling large scale optimization problems [45].

TS can be thought as an iterative descent method, starting with a randomly generated initial solution and examining the neighborhood around it. However, in contrast to classical descent methods, for example the steepest descent method, where the algorithm terminates as soon as the solution stops improving, in the TS algorithm there is no such a restriction: non-improving solutions can be accepted, in order to avoid being trapped in local minima. Therefore, in the latter, the process continues from the best solution found in the current neighborhood, while the best solution found so far is stored. It must be noted, however, that in order to avoid revisiting previously visited solutions, the algorithm makes use of a list, known as tabu list (TL). The TL contains the history of the search and is characterized as the most distinctive feature of tabu search [46]. This list has a fixed size $|TL|$ and is updated by adding the last moves to the beginning of the list and discarding the oldest elements, if necessary. All the moves back to solutions being stored in the TL are called tabu moves and are forbidden for the next $|TL|$ iterations. The size of the TL affects the search process in the sense that increased sizes result in a more likely rejection of a new solution, leading the search in areas which have not recently been tested. Tabu status can be recalled, provided that a condition known as aspiration criterion (AC), is satisfied. The AC allows the algorithm to perform moves even if they have been characterized as tabu. For example, a tabu move can lose its tabu status under the condition that it yields a solution with better objective function value than the best found so far; this condition is known as aspiration by objective.

The above description concerns the standard TS procedure, over which many improvements have been introduced in the literature [47]. These modifications may include multiple aspiration criteria and tabu lists, variable tabu list sizes, systematic intensification and diversification strategies for neighborhood creation, long term memory implementation and many others, depending on the individual problem at hand. In this work, the basic TS algorithm is adapted so as to facilitate the search for a fuzzy partitioning which results to models with increased accuracy.

4.2. Proposed method

At each iteration i of the proposed TS-NSFM procedure, the elements of the solution S_i correspond to the number of fuzzy sets in each dimension

$$\mathbf{S}_i = [s_1, s_2, \dots, s_N]^T \quad (8)$$

as for example is shown in Fig. 3 where a 3-D input space is partitioned in 4, 3 and 6 fuzzy sets in each dimension, respectively. The algorithm starts by generating a random solution $\mathbf{S}_0 \in \mathbb{R}^N$ within a specific range $[s_{\min}, s_{\max}]$, where $s_{\min}$ and $s_{\max}$ is the minimum and maximum number of fuzzy sets, respectively, in each dimension. Given a solution $\mathbf{S}_i$ at each iteration i , the number of RBF units and the locations of their centers are determined using the standard NSFM algorithm. After fixing the parameters of the RBF units, the synaptic weights can be calculated using (7), resulting to a fully trained RBF network. The next step is to evaluate the performance of the network, typically using an error-based cost function F .

One crucial step in the TS algorithm is the determination of the neighborhood construction, as its examination leads the search process. In this case, as the number of fuzzy sets can receive only integer values, the neighborhood of a solution $\mathbf{S}_i$ is defined as $\mathbf{S}_i \pm m$, where m is a positive integer ($m < s_{\min}$). Thus, the neighborhood is defined as a set of integer points on the edges of an N -dimensional hyper-square within the search space.

In Fig. 4, an example of neighborhood selection is given using a simple 2-D case for visualization purposes. In this example, $m = 2$ is chosen and, given an initial solution $\mathbf{S}_0 \in \mathbb{R}^2$, its neighborhood is constructed as a 2-D square around $\mathbf{S}_0$. The points of the neighborhood can be classified into the L set, which contains the points lying in the edges of the square (symbolized by circles) and the N set, which contains the points lying in the face of the square (symbolized by crosses). The proposed algorithm employs two TLs, namely, a short-term TL and a long-term TL. The former has a fixed size $shortTL$; at each iteration, all the neighborhood points of the current solution (all the points of L and N set) are added to the beginning of this list. The latter contains all the visited solutions $\mathbf{S}_i$. The main difference between the two lists lies in the fact that an element of the long-term TL can never be visited again, while an element of the short-term TL can lose its tabu status, either when a number of iterations pass (by fixing the TL size), or when the

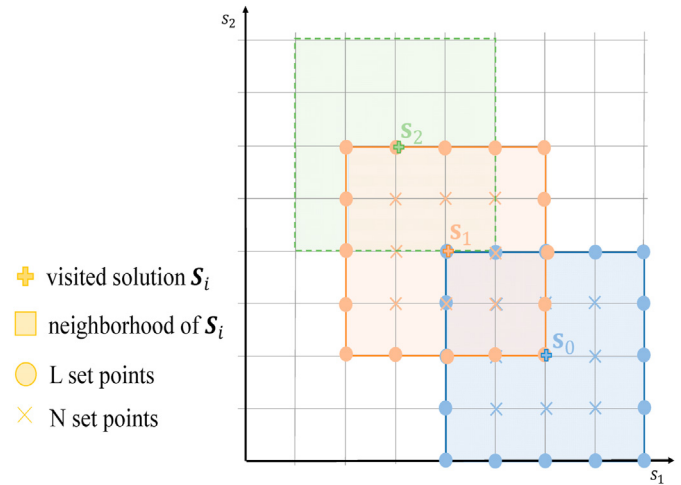


Fig. 4. Illustration of the neighborhood creation in a 2-D problem.

aspiration by objective holds.

Starting with the first iteration and given that both tabu lists are empty, all neighborhood points are candidates to become the next solution $\mathbf{S}_1$. After the objective function has been evaluated on these points, the next solution $\mathbf{S}_1$ is defined as the best solution found in the neighborhood of $\mathbf{S}_0$ and the process continues by defining a new square around $\mathbf{S}_1$. Before proceeding to the next iteration, the solution $\mathbf{S}_0$ is added to the long-term TL and its neighborhood is added to the beginning of the short-term tabu list (fixing its size when needed). Therefore, a set which contains the solutions which are not classified as tabu, or are tabu but the aspiration criterion holds for them, is created, namely the V^* set. However, if all available moves are classified as tabu and the AC is not valid, the V^* set will be empty. In this case, an aspiration by default [48] is used: the solution that gives the least increase in the objective function of the current iteration is selected. The

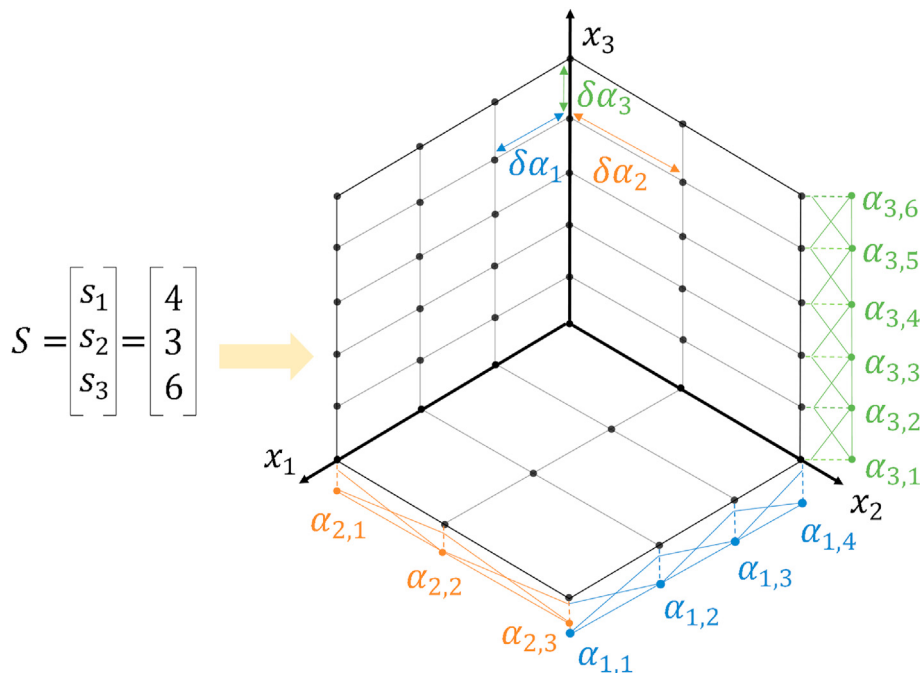


Fig. 3. A non-symmetric fuzzy partitioning of a 3-D input space: the number of fuzzy sets in each dimension presents the corresponding element of the solution.

neighborhood of S_1 is constructed in the same way, however, the available moves are less than before, because some points are classified as tabu and the aspiration criterion must be checked for the latter in order to be accepted. The aspiration criterion concerns the objective function value: a solution being tabu can be accepted provided that it yields better objective than the global best solution. The search continues from S_2 , which is the best solution found in the neighborhood of S_1 (coming from the V^* set), and the algorithm proceeds in this way, until a termination condition is met. In this work, a maximum number of iterations is used as a termination criterion. Algorithm I presents an overview of the TS-NSFM algorithm.

It is noted that the value of m controls the size of the constructed neighborhood. The solution neighborhood increases with m , performing a broader search within the search space. However, large values of m in a high dimensional problem may be impractical, because the algorithm cannot focus in a specific area of the search space (poor exploitation). Moreover, whenever the global best solution is updated, the neighborhood shrinks to $m = 1$ for the next iteration; this is a modification that allows the algorithm to intensify local search around a newly discovered global best solution. The value of m is increased again at the next iteration, allowing for better exploration of the search space.

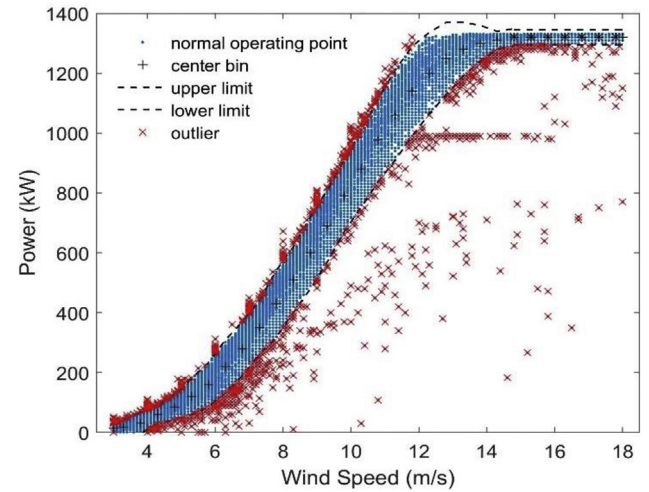


Fig. 5. Scatter power curve for wind turbine no. 4. Red cross points show points that were filtered out, while blue dots show data points marked as normal operation points. (For interpretation of the references to colour in this figure legend, the reader is referred to the Web version of this article.)

Algorithm I – TS-NSFM

Input: $\{x, y\}$: dataset,
 $[S_{min}, S_{max}]$: minimum and maximum number of fuzzy sets,
 m : neighborhood size parameter,
 TL : tabu list size parameter
 $MaxIter$: maximum number of iterations

Output: c : RBF center locations
 w : RBF network weights

- 1: generate a random initial solution $S_0 \in \mathbb{R}^N$ drawn from $[S_{min}, S_{max}]$,
- 2: pass S_0 and the dataset to the NSFM algorithm,
in order to calculate the number of RBF units and the locations of centers c_0 ,
- 3: calculate synaptic weights w using (7),
- 4: calculate cost function $F_0(S_0)$,
- 5: set the initial solution as global best solution:
 $S^* = S_0, F^* = F_0, i^* = i_0$
- 6: start with the first iteration: $i \leftarrow 1$
- 7: **While** the termination condition is not satisfied **Do**:
- 8: Construct the neighborhood of S_i , ($S_i \pm m$)
- 9: Apply NSFM algorithm to return c, w
Compute $F_i(S_i \pm m)$ at L set
- 10: Find $S_{i+1} \leftarrow \underset{S_i \pm m \in V^*}{\operatorname{argmin}} F(S_i \pm m)$
where $V^* = \{\text{not in TL or AC holds}\}$
- 11: **If** V^* set is empty
apply AC by default
- 12: **End If**
- 13: Update tabu lists
- 14: Fix TL size, if needed
- 15: **If** $F(S_{i+1}) < F(S^*)$
- 16: Update the best solution
 $S^* \leftarrow S_{i+1}, F^* \leftarrow F_{i+1}, i^* \leftarrow i$
Set $m = 1$ for the next iteration
- 17: **End If**
- 18: Proceed to the next iteration: $i \leftarrow i+1$
- 19: **End while**

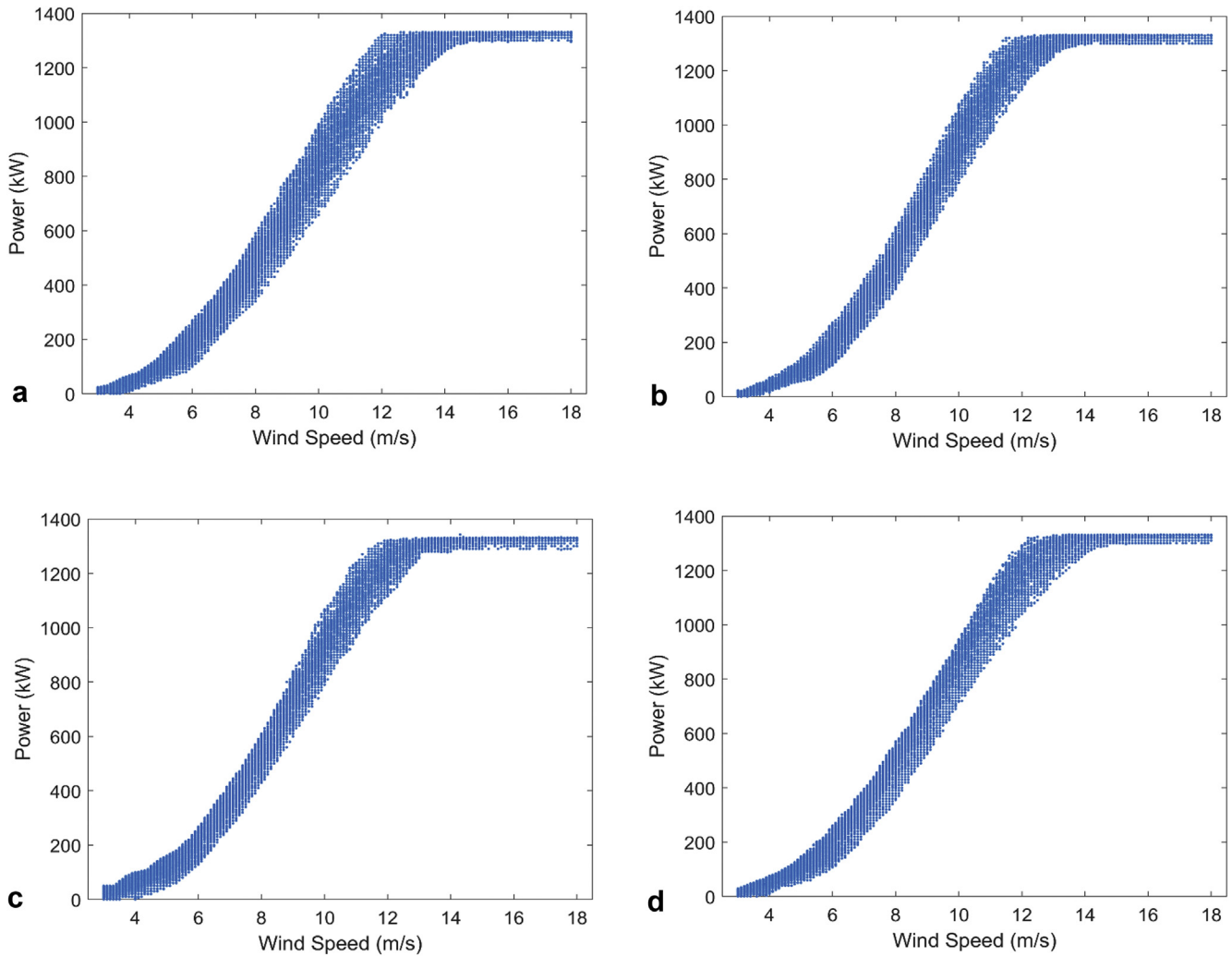


Fig. 6. Filtered power curves for: (a) wind turbine no. 1, (b) wind turbine no. 2, (c) wind turbine no. 3 and (d) wind turbine no. 4.

Table 1

Number of wind turbine datapoints before and after filtering.

Dataset	No. of examples <i>before</i> filtering	No. of examples <i>after</i> filtering
Wind turbine no. 1	49,476	34,262
Wind turbine no. 2	49,764	34,893
Wind turbine no. 3	49,764	32,428
Wind turbine no. 4	49,736	33,690

5. Case study

5.1. Data pre-processing

Before applying the proposed method, the available datasets have to be cleared out from abnormal operation points that are defined as outliers. To this purpose, a filtering stage is applied in order to improve the quality of the datasets, so that normal operation points will be mainly used for power curve modeling. In literature, various filters for outlier detection in the power curve of a wind turbine have been applied [21,49–52].

In this work, the filtering procedure is adapted from Ref. [52] and is based on power output. The process is as follows: the wind speed range is first partitioned into wind speed bins of width equal

Table 2

Operational parameters for the TS-NSFM algorithm.

Parameter	Symbol	Value/Description
Minimum number of fuzzy sets	$s_{\min}$	5
Maximum number of fuzzy sets	$s_{\max}$	60
Neighborhood size ^a	m	Updated best: 1, otherwise: 2
Tabu List size	shortTL	2500
	longTL	∞
Maximum number of iterations	P	80

^a Value depending on finding a cost solution better than the best found so far.

to 0.5 m/s. For each bin, the median value and the median absolute deviation (MAD) for power output are calculated. The MAD is defined as

$$MAD = \text{median}(|X_i - \tilde{X}|) \quad (9)$$

where $\tilde{X}$ is the median value. The center of each bin is represented by the median value of wind speed and the median value for power output. In each wind speed bin, the power output points that are outside of $\pm 3MAD$ are marked as outliers and are discarded. Fig. 5 shows an example of applying this filtering for wind turbine no. 4. It is noted that the MAD method is suggested for outlier detection

[53] as a robust statistic and is used here so that the calculation is not biased by a few outliers [52].

The filtered scattered power curves for each wind turbine are depicted in Fig. 6 considering a wind speed range from 3.5 to 18 m/s, which are the cut-in and the cut-out wind speeds, respectively. After the filtering procedure is applied, the number of remaining data points for each dataset is depicted in Table 1.

Following the filtering stage, the available data should be divided randomly in three independent subsets, namely the training, validation and testing subset. The training subset is used in the training process, i.e. the process where the network parameters are fitted to the data. In order to avoid over-fitting to the training subset (a situation where the model lacks the ability of generalization, i.e. to provide successful estimations for data points different than the ones used for training), the validation subset is used for model selection. Finally, the testing subset, which is not presented to the network during the training and model selection stages, is used for an unbiased evaluation of the model. In this study, the available data for each wind turbine are divided randomly using a 50% - 25% - 25% ratio for the training - validation - testing subsets, respectively. Random splitting of the filtered data ensures a balanced distribution of data points over the three datasets, in the corresponding period of one year.

5.2. Algorithm implementation

The parameters used by the TS-NSFM algorithm are presented in Table 2. The lower and the upper bounds of the search space are denoted as $s_{\min}$ and $s_{\max}$, i.e. the minimum and maximum number of fuzzy sets, respectively; the values for $s_{\min}$ and $s_{\max}$ are selected based on recommendations from other studies [31].

The value of m controls the size of the constructed neighborhood, thus its choice affects the search process, favoring either exploitation, or exploration of the solution space. After performing a trial and error procedure, a value of $m = 2$ was found to strike a satisfactory balance between exploration and exploitation for all datasets.

The size of the short-term TL is a parameter that also affects the exploration capabilities of the method, because, as it increases, the algorithm is prohibited to return to recently visited neighborhoods and thus tends to test new solutions. A list size of 2500 is selected, which, in conjunction with the selected value of m , implies that the short-term TL is refreshed every 4 iterations, approximately.

Model performance is evaluated based on the mean absolute error (MAE) which is calculated as follows:

$$MAE = \frac{1}{K} \sum_{i=1}^K |y_i - \hat{y}_i| \quad (10)$$

where y_i and $\hat{y}_i$, ($i = 1, 2, \dots, K$) are the real measurements and model estimations, respectively.

For comparison purposes, a variety of additional methodologies including parametric and non-parametric models, as well as the manufacturer power curve, were also tested in order to investigate the effectiveness of the proposed model.

The first non-parametric method concerns an RBF network trained with the SFM algorithm. The SFM algorithm is used to provide an indication of how the proposed approach compares to a standard, well-tested RBF training method. Model selection for SFM is controlled by a single parameter, i.e. the number of fuzzy sets s which, for the symmetric case is the same for every input variable. The optimal value of s was found by following an exhaustive search procedure in the range [5–60], using the validation subset.

The second non-parametric approach was selected so as to test

against a different neural network architecture. To be more specific, the MLP network architecture was chosen and three different training algorithms were implemented, leading to three variants of the MLP approach. In all MLP variants and in order to ensure a fair comparison with the RBFN, the network parameters were determined using the training subset and model selection was performed using the validation subset. The testing subset was used for an unbiased evaluation of the model performance. A two hidden layer structure was postulated for all MLP networks, and an exhaustive search procedure was used to determine the optimal number of neurons for each layer, testing the range [10–20] for the first hidden layer and the range [5–15] for the second.

More particularly, the first variant concerns an MLP network trained using Levenberg-Marquardt back-propagation [54]; the particular configuration was selected as it constitutes one of the most popular methods for ANN training. The second variant concerns an MLP network trained with a state-of-the-art stochastic gradient-based optimization technique called ADAM, an approach that is proposed for WTPC modeling in Ref. [22]. The third variant concerns an MLP network trained with the back-propagation algorithm, while using a genetic algorithm to improve the performance of the network, as described in Ref. [24]. In all the cases of MLP variants, the training algorithms were tuned following the suggestions given in the respective publications.

Due to their stochastic nature, the TS-NSFM algorithm and the three MLP variants produce a different result for each run; thus, 10 different runs were performed for each case, in order to test the consistency of the results.

The last non-parametric model concerns a cubic spline interpolation technique [24], which is based on piecewise polynomial regression models. The form of each cubic spline in each sub-interval of $v_0 < v_1 < \dots < v_{\max}$ is in the form of

$$\hat{p}(v) = av^3 + bv^2 + cv + d \quad (11)$$

where $\hat{p}$ is the power estimated by the model (in kW), v is the wind speed (in m/s), and the coefficients a, b, c, d are calculated by using least squares.

The parametric model was taken from Ref. [11] and is given by:

$$\hat{p}(v) = p_{\max} \left(1 + \left(\frac{\beta}{v} \right)^{\alpha} \right)^{-\kappa}, \quad \alpha, \beta, \kappa > 0 \quad (12)$$

where $p_{\max}$ is the maximum power output of wind turbine (in kW) and α, β, κ are parameters to be fitted to the data. To be more specific, these parameters were estimated by solving the following nonlinear least squares problem:

$$\min_{\alpha, \beta, \kappa} \sum_{i=1}^n \left(p_i(v) - \hat{p}_i(v) \right)^2 \quad (13)$$

In both cubic spline regression and parametric model, in order to ensure a fair comparison with the rivaling methods, the training and the validation subsets were used for fitting the parameters, whereas the testing subset was used for evaluating model performance.

To conclude the comparison, the power versus wind speed curve provided by the manufacturer is also employed as a model. In this case, the estimation of generated power is obtained directly from the power curve, based solely on the observed wind speed.

5.3. Results and discussion

In Fig. 7, the proposed model power output estimations are compared to the actual data from the testing subset, for each wind

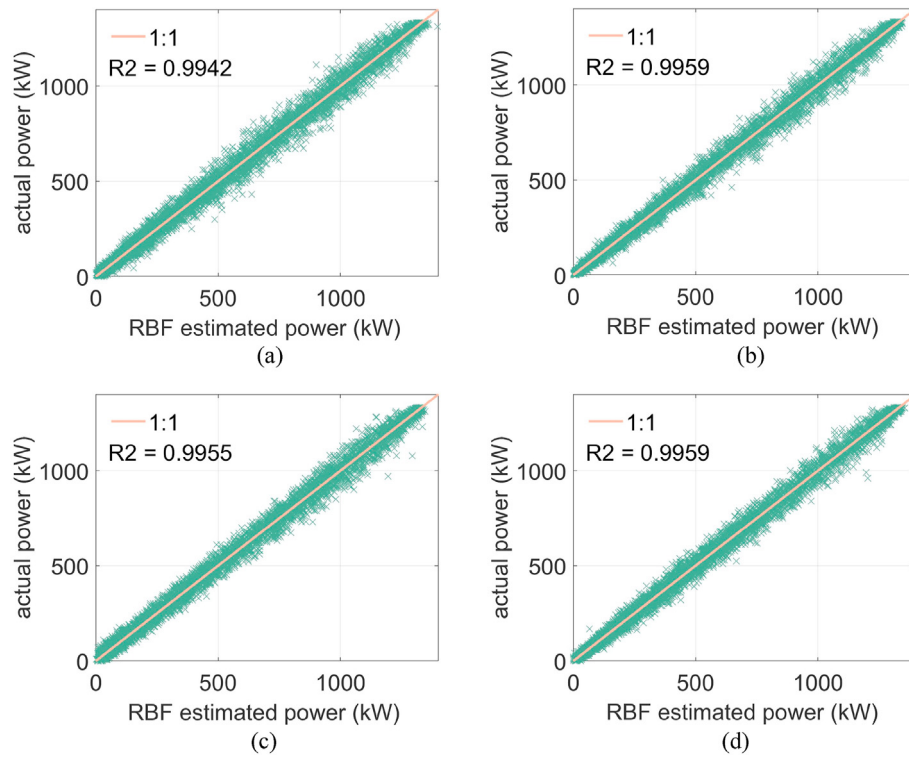


Fig. 7. Actual vs. RBF network estimated power output for: (a) wind turbine no. 1, (b) wind turbine no. 2, (c) wind turbine no. 3 and (d) wind turbine no. 4, regarding the testing subset. The identity line $y = x$ (1:1 line) is also depicted as reference.

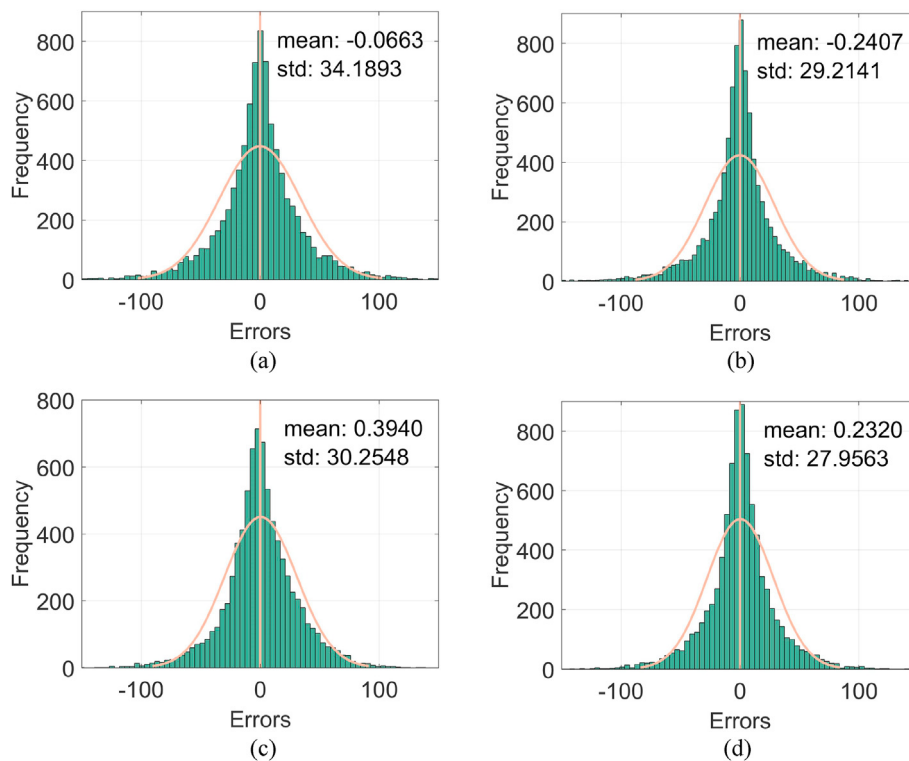


Fig. 8. Error frequency distribution for: (a) wind turbine no. 1, (b) wind turbine no. 2, (c) wind turbine no. 3 and (d) wind turbine no. 4. The normal distribution fit (orange line) is also depicted as reference. (For interpretation of the references to colour in this figure legend, the reader is referred to the Web version of this article.)

Table 3

Results for all wind turbines – MAE for testing subset, fuzzy partition, number of nodes, number of iterations and computational time.

Wind turbine	Algorithm	MAE testing (kW)	Fuzzy partition	No. of nodes	Iterations	Computational time (s)
1	TS-NSFM	23.6915 (23.7403 ± 0.1186)	[45 46 50 50]	1835 (1839 ± 127)	27 (53 ± 15)	4896 (5189 ± 981)
	SFM	23.8593	47	1790	—	987
	MLP - LM	24.7542 (24.7704 ± 0.7618)	—	[13,15]	—	12,325 (11,139 ± 1035)
	MLP - ADAM	24.3149 (26.5116 ± 2.0151)	—	[13,19]	—	6231 (9832 ± 861)
	MLP - GA	24.6219 (25.6833 ± 3.2316)	—	[5,20]	—	9586 (10,239 ± 1351)
	Cubic spline	31.8234	—	—	—	1.50
	Parametric	34.4718	—	—	—	1.85
	Manufacturer Curve	35.3281	—	—	—	—
2	TS-NSFM	19.6200 (19.6639 ± 0.0818)	[44 50 28 48]	1706 (1475 ± 218)	60 (54 ± 15)	3220 (4529 ± 622)
	SFM	19.7357	46	1658	—	1002
	MLP - LM	20.2913 (20.1629 ± 0.3678)	—	[10,18]	—	9692 (9320 ± 891)
	MLP - ADAM	20.2396 (21.1101 ± 1.1662)	—	[12,13]	—	9791 (8632 ± 1320)
	MLP - GA	20.2334 (20.9871 ± 3.9126)	—	[6,20]	—	10,294 (9483 ± 1183)
	Cubic spline	24.7718	—	—	—	1.31
	Parametric	26.5341	—	—	—	1.46
	Manufacturer Curve	50.1988	—	—	—	—
3	TS-NSFM	21.7047 (21.7809 ± 0.0785)	[50 32 29 47]	1376 (1223 ± 197)	60 (47 ± 14)	4543 (4150 ± 1107)
	SFM	21.8782	46	1635	—	877
	MLP - LM	21.8993 (21.5560 ± 0.5932)	—	[9,17]	—	10,691 (9296 ± 769)
	MLP - ADAM	22.4449 (23.6625 ± 1.0168)	—	[13,19]	—	5623 (9092 ± 964)
	MLP - GA	23.2192 (24.1656 ± 3.4673)	—	[6,18]	—	9732 (10,139 ± 1241)
	Cubic spline	25.6926	—	—	—	1.28
	Parametric	26.9326	—	—	—	1.83
	Manufacturer Curve	53.3621	—	—	—	—
4	TS-NSFM	19.3407 (19.3063 ± 0.0704)	[24 47 19 46]	1010 (1059 ± 109)	23 (54 ± 19)	3161 (3592 ± 804)
	SFM	19.3950	43	1451	—	842
	MLP - LM	19.4315 (19.4557 ± 0.0755)	—	[11,20]	—	10,645 (9830 ± 967)
	MLP - ADAM	19.9953 (20.8257 ± 1.3126)	—	[13,17]	—	7283 (8432 ± 1052)
	MLP - GA	20.5242 (21.4176 ± 2.7471)	—	[5,14]	—	6893 (9291 ± 938)
	Cubic spline	26.8889	—	—	—	1.06
	Parametric	28.6257	—	—	—	1.54
	Manufacturer Curve	30.6139	—	—	—	—

^a The TS-NSFM algorithm and the optimal configuration for the MLP models were tested 10 times for each wind turbine dataset. The best result (corresponding to the run achieving the lowest MAE in the validation set), followed by the average and standard deviation values from the 10 runs in parentheses is depicted. The highest performance achieved for each wind turbine dataset is shown in boldface type.

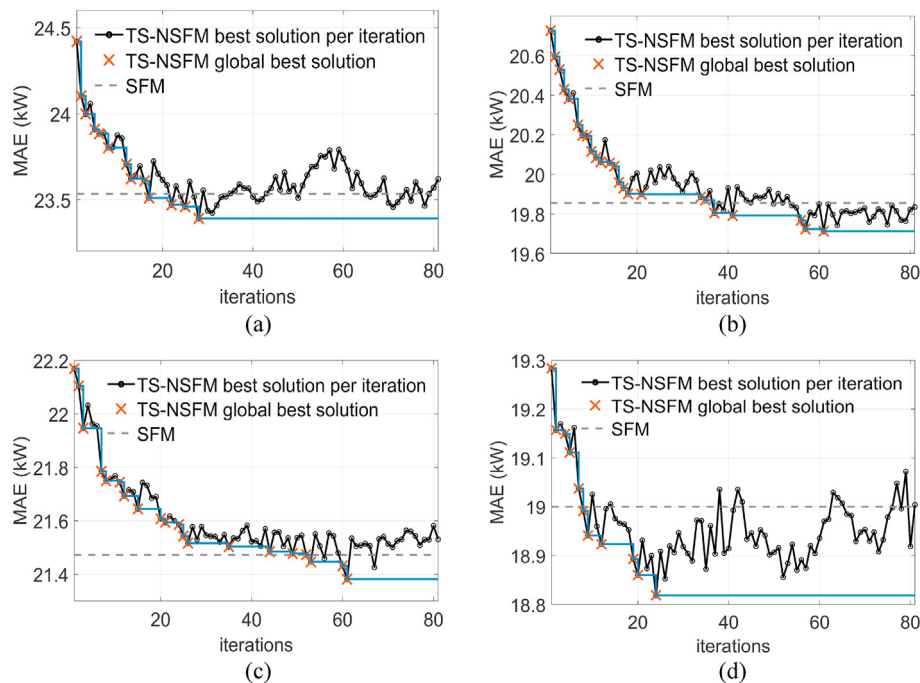


Fig. 9. Evolution of MAE per iteration in the validation subset for the TS-NSFM algorithm: (a) wind turbine no. 1, (b) wind turbine no. 2, (c) wind turbine no. 3 and (d) wind turbine no. 4.

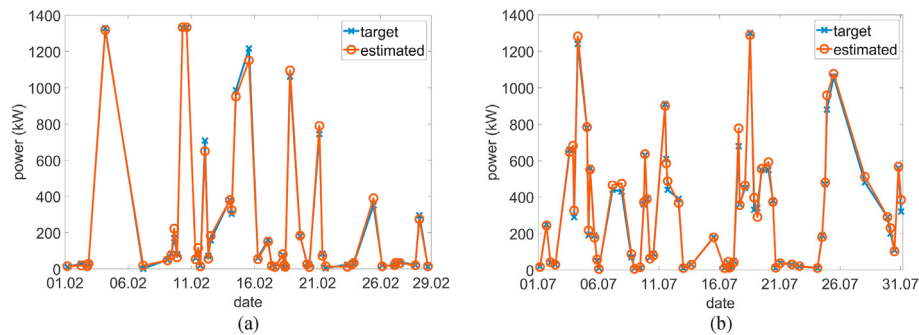


Fig. 10. Estimations of power output in the testing subset for the TS-NSFM algorithm in (a) winter and (b) summer periods. Note that for visualization purposes, only a random subset of the testing data is being displayed.

turbine. The RBF network estimations fit well to the actual data, with the calculated values of the coefficient of determination (R^2) being over 99.4% for all WTs.

The error frequency distributions regarding the testing subset for each wind turbine case are shown in Fig. 8. The error histograms appear to be symmetric around zero, attesting to the accuracy of the proposed model. The standard deviation values represent approximately 2.6% of wind turbine maximum power.

The results from the application of TS-NSFM algorithm in comparison with the rest of the competing methods for all wind turbines are summarized in Table 3, depicting the MAE values in the testing subset, followed by the selected fuzzy partitioning, the number of RBF nodes, the iterations needed to find the best network configuration and the total computational time for each run. All computational times were measured on a PC with Intel Core i5-4690 K CPU and 12 GB DDR3 RAM. The results concerning the TS-NSFM algorithm and the three MLP variants represent the best run, i.e. the run achieving the lowest MAE in the validation set, followed by the average and standard deviation values of the 10 runs in parentheses, for each wind turbine dataset.

As shown in Table 3, the TS-NSFM algorithm clearly outperforms all its rivals for all WTs in terms of accuracy, using either the best of the 10 runs, or their average as a basis for comparison. It is also noted that standard deviations of MAE values were found to be relatively small, which is an indication that the proposed approach produces consistent results. Regarding the comparison with the standard SFM algorithm, this result is due to the fact that in TS-NSFM the number of fuzzy sets is allowed to be different for each dimension. This provides greater flexibility in center selection and, in conjunction with the proposed TS method for optimizing the fuzzy partitioning, eventually results in better estimation accuracy for the generated power. As the size of the constructed network is concerned, it is important to note that in 3 out of 4 cases, the TS-NSFM algorithm reduces the number of nodes when compared to SFM, thus providing simpler, and at the same time more accurate, models; for example, the model size obtained by TS-NSFM algorithm for wind turbine no. 4 corresponds to a reduction of over 30% compared to SFM.

Fig. 9 provides an insight on how the proposed algorithm manages to reduce modeling error, by depicting the evolution of MAE in the validation subset per iteration for the best run of the TS-

NSFM algorithm. The black line with circles stands for the accepted solution for each iteration, which is not always improving; indeed, as it was explained in section 4, the TS technique may result in non-improving solutions to be temporarily accepted, in order to escape from poor local minima. This is confirmed by the evolution of the blue line with crosses, which stands for the best solution found by the algorithm up to the current iteration, and is always decreasing, signifying the improvement of the produced models.

As far as computational times are concerned, it is clear that the TS-NSFM algorithm takes more computational time to complete the search task, compared to the SFM training method. This is expected because the former needs to explore a significantly larger search space, but it does not necessarily mean that SFM is indeed more computationally efficient. This can be explained by examining Fig. 9, which, besides the evolution of the TS-NSFM solutions, also depicts the best solution found by the simple SFM algorithm. It can be seen that the TS-NSFM algorithm starts from a solution with significantly higher MAE; this is expected due to the random choice of the initial solution. However, TS-NSFM gradually decreases the modeling error and in fact it manages to outperform SFM only within a small number of iterations. This means that, although TS-NSFM invariably takes more time to complete the whole search procedure, in most cases it is able to discover a better solution compared to SFM in lower computational times than the latter.

Compared to the MLP network variants, it can be seen that in all cases the TS-NSFM algorithm outperforms all MLP solutions in terms of both best and average MAE values in the testing subset, while also resulting to lower standard deviation values. Although both RBF and MLP are feed-forward networks, the simple structure of the former enables efficient computation of its parameters; for instance, weights can be computed through linear least squares, and, in conjunction with the proposed TS-NSFM algorithm, a fully trained RBF network resulting to superior accuracy for power curve modeling emerges. Narrowing the comparison to the MLP models, their results seem to be comparable. This can be explained as MLP-LM and MLP-ADAM use state-of-the-art training techniques, while MLP-GA on the other hand employs simple backpropagation for training, but is enhanced with a genetic algorithm which improves parameter optimization.

A comparison between the TS-NSFM algorithm and the MLP variants in terms of computational times, shows that the former

Table 4
MAE for the testing subset during specific periods of the year.

Method	TS-NSFM	SFM	MLP-LM	MLP-ADAM	MLP-GA	Cubic spline	Parametric	Manufacturer Curve
MAE (kW) February	18.9955	19.1008	19.2927	21.2791	21.3418	32.2441	31.1267	41.8630
MAE (kW) July	18.9325	19.0152	19.1313	19.4893	20.0199	31.7183	33.5796	26.0229

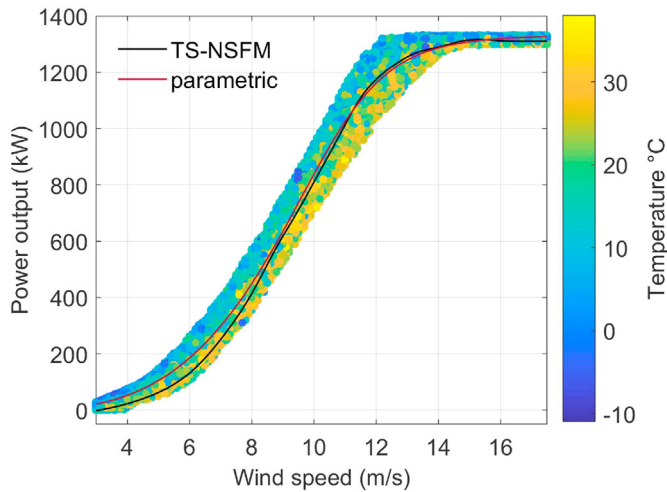


Fig. 11. WTPC model comparison between TS-NSFM and parametric method using univariate analysis.

accomplishes the training task significantly faster. This highlights an important contribution of the proposed approach, which, in contrast to the particular MLP variants, avoids tedious exhaustive search procedures and population-based search methods and thus manages to reduce the computational time needed for training, while at the same time providing more accurate models in all cases.

Cubic spline interpolation and the parametric function used for power output estimation have a fixed form, a number of parameters to be estimated and, in contrast to the ANN models, only one input, i.e. the wind speed. Spline regression leads to better results over the parametric model, as it provides more flexibility by fitting more free parameters to the data. However, due to the fact that power generation by a wind turbine depends on various factors besides wind speed, it is reasonable for the ANN models to produce higher accuracy. This advantage gained by the ANN models in terms of accuracy when compared to cubic spline interpolation and the parametric model, is significant enough to justify the considerable increase in computational times needed by the formers. This increase can be attributed to the fact that both cubic spline interpolation and the parametric model use much simpler algorithms for fitting compared to the ANNs, while they do not involve any

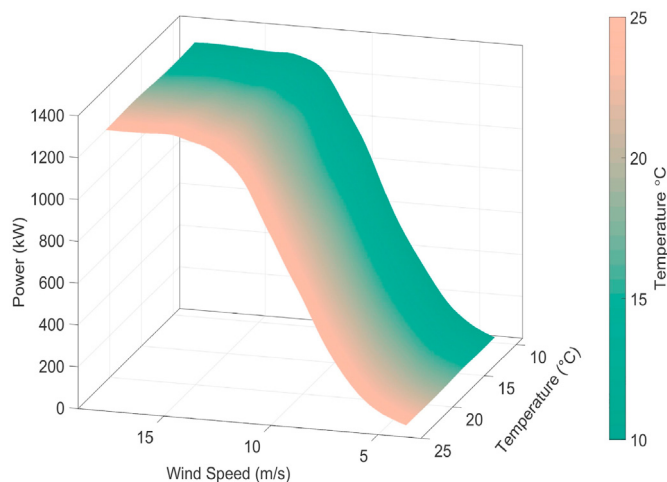


Fig. 12. 3-D power curve for wind turbine no. 4 at specific conditions regarding blade pitch and wind direction angles.

structure optimization procedure.

Finally, the manufacturer curve model is inferior to the rest of the power curve modeling techniques in terms of MAE values in the testing subset. This result is well in accordance with the literature [6], and is due to the fact that the manufacturer curve is obtained under rather ideal and specific conditions, thus ending in inaccurate results when employing it as a model to estimate power output in actual site conditions, where many factors affect the generated power.

The effectiveness of the proposed model is further evaluated by employing it for accomplishing certain tasks, as shown in the next paragraphs; wind turbine no.4 is selected as a case study for all the accomplished tasks.

- Model results for shorter periods of the year** In order to assess the modeling accuracy of the proposed approach during specific periods of the year, its performance was evaluated separately on typical winter and summer periods, containing the testing data for the months of February (657 data points) and July (785 data points), respectively. Fig. 10 depicts the target generated power values and the ones estimated by the TS-NSFM approach, where it can be seen that the estimations track accurately the real values for both months. Table 4 gives the MAE values of the best model produced by each different method, as far as the testing subset is concerned. As it can be seen, the proposed TS-NSFM algorithm clearly outperforms its rivals for both seasons of the year.

- Obtaining 2-D power curves under given conditions** The objective in this case is to evaluate the potential of the proposed method to obtain 2-D power curves under given conditions. In particular, Fig. 11 illustrates a comparison between the power curve obtained by the proposed TS-NSFM algorithm and the parametric model. To generate the WTPC used for comparison, values near the median of the measured data have been employed: 24 °C, 0° and 98°, for temperature, blade pitch angle and wind direction, respectively. The power curve obtained by the proposed model provides notably better results in approximating the power output, while the parametric model seems to overestimate it, especially in high wind speeds. The superiority of the former is based on its flexibility to consider several variables that influence the power output at the same time, in contrast to the parametric model.

- Obtaining 3-D power curves under given conditions** In Fig. 12, a typical 3-D visualization of the power output as a function of wind speed and temperature is depicted. The 3-D surface is constructed based on the TS-NSFM model estimations with fixed wind direction and blade pitch angles at 98° and 0°, respectively. It can be easily observed that the power output increases as the wind speed increases in the entire temperature range. Moreover, a slight decrease to the power output can be clearly seen as the temperature increases, confirming the dependence of the produced power on the ambient temperature.

6. Conclusions

This paper introduces a new method for producing highly accurate power curve models of wind turbines, by taking into account other important variables besides wind speed, namely the wind direction, ambient temperature and blade pitch angle in this case. The method concerns the development of a non-parametric model and, in particular, an RBF neural network, which is characterized by

its simple architecture. The network parameters are determined through a novel training procedure based on the NSFM algorithm, which presents greater flexibility in parameter selection compared to the standard symmetric version, thus producing networks of increased accuracy. As the increased number of inputs and the significant amount of data which are needed for training a model of high accuracy render the use of exhaustive search, or even population-based metaheuristic search methods impractical, we propose a novel approach based on a modification of the TS metaheuristic technique, which results to the TS-NSFM algorithm. To evaluate the performance of the proposed algorithm, a set of experiments was conducted using four wind turbine datasets and, in order to further demonstrate the efficiency of the modeling approach, a number of parametric and non-parametric modeling methods, including three different MLP network-based schemes, were included for comparison. The comparative results are in favor of the TS-NSFM algorithm, as it achieves to produce models with higher accuracy compared to all its rivals, while also outperforming the other ANN approaches in terms of shorter computational times. Moreover, the proposed models can be applied for building wind turbine performance analysis tools, as confirmed by the produced graphs in both 2-D and 3-D cases, constructed based on the model estimations.

Future research plans include applying the proposed algorithm to address other challenging applications in wind energy systems, such as forecasting, or fault detection and diagnostics; taking advantage of the remarkable features of RBFN, it is expected that flexible models with increased accuracy can be developed in shorter computational times.

CRediT authorship contribution statement

Despina Karamichailidou: Data curation, Formal analysis, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing - original draft, Writing - review & editing. **Vasiliki Kaloutsas:** Data curation, Formal analysis, Investigation, Methodology, Resources, Validation, Writing - original draft, Writing - review & editing. **Alex Alexandridis:** Conceptualization, Formal analysis, Methodology, Project administration, Resources, Supervision, Validation, Writing - original draft, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] V. Nelson, *Wind Energy: Renewable Energy and the Environment*, second ed., Press, CRC, 2013.
- [2] J.F. Manwell, J.G. McGowan, A.L. Rogers, *Wind Energy Explained: Theory, Design and Application*, 2nd ed., John Wiley & Sons, Ltd, 2009.
- [3] T. Burton, N. Jenkins, D. Sharpe, E. Bossanyi, *Wind Energy Handbook*, second ed., John Wiley & Sons, Ltd, 2011.
- [4] M. Jafarian, A.M. Ranjbar, Fuzzy modeling techniques and artificial neural networks to estimate annual energy output of a wind turbine, *Renew. Ener.* 35 (2014) 2008–2010.
- [5] S. Frandsen, et al., Redefinition power curve for more accurate performance assessment of wind farms, *Wind Ener.* 3 (2000) 81–111.
- [6] S. Shokrzadeh, M. Jafari Jozani, E. Bibeau, Wind turbine power curve modeling using advanced parametric and nonparametric methods, *IEEE Trans. Sustain. Ener.* 5 (4) (2014) 1262–1269.
- [7] M. Lydia, S. Selvakumar, A. Immanuel Suresh Kumar, G. Edwin Prem Kumar, Advanced algorithms for wind turbine power curve modeling, *IEEE Trans. Sustain. Ener.* 4 (3) (2013) 827–835.
- [8] M. Lydia, S. Suresh Kumar, A. Immanuel Selvakumar, G. Edwin Prem Kumar, A comprehensive review on wind turbine power curve modeling techniques, *Renew. Sustain. Ener. Rev.* 30 (2014) 452–460.
- [9] V. Sohoni, S.C. Gupta, R.K. Nema, A critical review on wind turbine power curve modelling techniques and their applications in wind based energy systems, *J. Ener.* 2016 (2016).
- [10] V. Thapar, G. Agnihotri, V.K. Sethi, Critical analysis of methods for mathematical modelling of wind turbines, *Renew. Ener.* 36 (11) (2011) 3166–3177.
- [11] M. Marciukaitis, I. Zutauteite, L. Martisauskas, B. Jokas, G. Gecevicius, A. Sfetsos, Non-linear regression model for wind turbine power curve, *Renew. Ener.* 113 (2017) 732–741.
- [12] A. Kusiak, A. Verma, Monitoring wind farms with performance curves, *IEEE Trans. Sustain. Ener.* 4 (1) (2013) 192–199.
- [13] A. Engelbrecht, *Computational Intelligence: an Introduction*, second ed., John Wiley & Sons, Ltd, 2007.
- [14] V. Singh, Application of artificial neural networks for predicting generated wind power, *Int. J. Adv. Comput. Sci. Appl.* 7 (3) (2016) 250–253.
- [15] N.J. Nilsson, *The Quest for Artificial Intelligence*, Cambridge University Press, Cambridge, 2009.
- [16] A.P. Marugán, F.P.G. Márquez, J.M.P. Perez, D. Ruiz-Hernández, A survey of artificial neural network in wind energy systems, *App. Ener.* (2018).
- [17] A. Kusiak, H. Zheng, Z. Song, Models for monitoring wind farm power, *Renew. Energy* 34 (3) (2009) 583–590.
- [18] A. Marvuglia, A. Messineo, "Monitoring of wind farms' power curves using machine learning techniques, *Appl. Energy* 98 (2012) 574–583.
- [19] M. Schlechtingen, I. Santos, S. Achiche, Using data-mining approaches for wind turbine power curve monitoring: a comparative study, *IEEE Trans. Sustain. Ener.* 4 (3) (2013) 671–679.
- [20] F. Pelletier, C. Masson, A. Tahan, Wind turbine power curve modelling using artificial neural network, *Renew. Ener.* 89 (2016) 207–214.
- [21] T. Ouyang, A. Kusiak, Y. He, Modeling wind-turbine power curve: a data partitioning and mining approach, *Renew. Ener.* 102 (2017) 1–8.
- [22] B. Manobel, F. Sehnke, J.A. Lazzús, I. Salfate, M. Felder, S. Montecinos, Wind turbine power curve modeling based on Gaussian Processes and Artificial Neural Networks, *Renew. Energy* 125 (2018) 1015–1020.
- [23] R.P. Shetty, A. Sathyabhama, P.S. Pai, Comparison of modeling methods for wind power prediction: a critical study, *Front. Ener.* (2018).
- [24] G. Ciulla, A. D'Amico, V. Di Dio, V. Lo Brano, Modelling and analysis of real-world wind turbine power curves: assessing deviations from nominal curve by neural networks, *Renew. Energy* 140 (2019) 477–492.
- [25] S. Haykin, *Neural Networks: A Comprehensive Foundation*, second ed., Prentice-Hall, 1999.
- [26] A. Alexandridis, E. Chondrodima, N. Giannopoulos, H. Sarimveis, A fast and efficient method for training categorical radial basis function networks, *IEEE Trans. Neu. Net. Learn. Syst.* 28 (11) (2017) 2831–2836.
- [27] G. Li, J. Shi, J. Zhou, Bayesian adaptive combination of short-term wind speed forecasts from neural network models, *Renew. Energy* 36 (1) (2011) 352–359.
- [28] G.W. Chang, H.J. Lu, Y.R. Chang, Y.D. Lee, An improved neural network-based approach for short-term wind speed and power forecast, *Renew. Energy* 105 (2017) 301–311.
- [29] Y. Han, S. Yang, Y. Liu, The comparison of BP network and RBF network in wind power prediction application. *Proceedings of Second International Conference on Bio-Inspired Computing, Theories and Applications*, 2007, pp. 173–176.
- [30] R.P. Shetty, P. Srinivasa, Pai, A. Sathyabhama, A. Rai, Optimized radial basis function neural network model for wind power prediction, in: *2016 Second International Conference on Cognitive Computing and Information Processing (CCIP)*, 2016, pp. 1–6.
- [31] A. Alexandridis, E. Chondrodima, H. Sarimveis, Radial basis function network training using a nonsymmetric partition of the input space and particle swarm optimization, *IEEE Trans. Neu. Net. Learn. Syst.* 24 (2) (2013) 219–230.
- [32] A. Alexandridis, H. Sarimveis, K. Ninos, "RBF network training using a nonsymmetric partition of the input space – application to an MPC configuration, *Adv. Eng. Softw.* 42 (10) (2011) 830–837.
- [33] A. Alexandridis, E. Chondrodima, A medical diagnostic tool based on radial basis function classifiers and evolutionary simulated annealing, *J. Biomed. Inf.* 49 (2014) 61–72.
- [34] F. Glover, Tabu search: part I, *OSRA J. Comput.* 1 (1989) 190–206.
- [35] J. Park, J. Lee, K. Oh, J. Lee, Development of a novel power curve monitoring method for wind turbines and its field tests, *IEEE Trans. Ener. Convers.* 29 (1) (2014) 119–128.
- [36] P. Guo, D. Infield, Wind turbine power curve modeling and monitoring with Gaussian process and SPRT, *IEEE Trans. Sustain. Ener.* 11 (1) (2020) 107–115.
- [37] D. Broomhead, D. Lowe, Multivariate functional interpolation and adaptive networks, *Complex Syst.* 2 (1988) 321–355.
- [38] K. Tao, A closer look at the radial basis function (RBF) networks, In *Proceedings of 27th Asilomar Conference on Signals, Syst. Comp.* (1993) 401–405.
- [39] S. Chen, C. Cowan, P. Grant, Orthogonal least squares learning algorithm for radial basis function networks, *IEEE Trans. Neu. Net.* 15 (2) (1991) 302–309.
- [40] C. Darken, J. Moody, Fast adaptive k-means clustering: some empirical results, in: *IEEE International Joint Conference. Neural Network*, 1990, pp. 233–238.
- [41] A. Alexandridis, M. Stogiannos, A. Kyriou, H. Sarimveis, An offset-free neural controller based on approximating the inverse process dynamics, *J. Proc. Contr.* 23 (7) (2013) 968–979.
- [42] A. Alexandridis, E. Chondrodima, E. Efthimiou, G. Papadakis, F. Vallianatos, D. Triantis, Large earthquake occurrence estimation based on radial basis

- function neural networks, *IEEE Trans. Geosci. Rem. Sens.* 52 (9) (2014) 5443–5453.
- [43] M. Stogiannos, A. Alexandridis, H. Sarimveis, Model predictive control for systems with fast dynamics using inverse neural models, *ISA Trans.* 72 (2018) 161–177.
- [44] J. Bezdek, *Pattern Recognition with Fuzzy Objective Function Algorithms*, Plenum press, New York (N.Y.), 1981.
- [45] A. Hertz, D. de Werra, The Tabu Search metaheuristic: how we used it, *Ann. Math. Artif. Intell.* 1 (1990) 111–121.
- [46] K.S. Al-Sultan, C.A. Fedjki, A tabu search-based algorithm for the fuzzy clustering problem, *Patt. Recogn.* 30 (12) (1997) 2023–2030.
- [47] B. Cao, F. Glover, C. Rego, A tabu search algorithm for cohesive clustering problems, *J. Heuristics* 21 (4) (2015) 1572–9397.
- [48] F. Glover, M. Laguna, Tabu search, in: D.-Z. Du, P.M. Pardalos (Eds.), in *Handbook of Combinatorial Optimization: Volume 1–3*, Springer US, 1998, pp. 2093–2229.
- [49] A. Llombart, C. Pueyo, J. Fandos, J. Guerrero, Robust data filtering in wind power systems, in: *European wind energy conference EWEC*, 2006.
- [50] P.E. Réthoré, N.A. Johansen, S.T. Frandsen, RBKS, Systematic wind farm measurement data reinforcement tool for wake model calibration, in: *European Offshore Wind Conference*, 2009, pp. 1–10.
- [51] M. Yesilbudak, Partitionial clustering-based outlier detection for power curve optimization of wind turbines", in: *IEEE International Conference on Renewable Energy Research and Applications*, 2016, pp. 1080–1084.
- [52] C. M. St Martin, J.K. Lundquist, A. Clifton, G.S. Poulos, S.J. Schreck, Wind turbine power production and annual energy production depend on atmospheric stability and turbulence, *Wind Energ. Sci.* 1 (2016) 221–236.
- [53] C. Leys, C. Ley, O. Klein, P. Bernard, L. Licata, Detecting outliers: do not use standard deviation around the mean, use absolute deviation around the median, *J. Exp. Soc. Psychol.* 49 (4) (2013) 764–766.
- [54] M.T. Hagan, M. Menhaj, Training feedforward networks with the Marquardt algorithm, *IEEE Trans. Neu. Net.* 5 (6) (1994) 989–993.